Dispatch

Assignment 3: MVP Implementation & Demo Report | Group Rapid Route (Mihir Shinde, Ananya)

Our Assignment 2 MVP was the traffic core: zone-density demand with an OD matrix and peak hours, car-following agents with safe gaps and junction right-of-way, and cost-based routing that can prefer a longer but faster path. All of it is built and runs in the browser (TypeScript, Vite, Three.js). The simulation core has no DOM access and is covered by 22 automated tests.

Plan vs. Reality

Made it in

Assignment 2 MVP item	Result
Zone density, OD matrix, peak and off-peak	Each zone has a density; a gravity OD matrix per time period (night, AM peak, midday, PM peak, evening) drives Poisson trip generation. Building heights follow density. The sidebar shows the OD heatmap, top OD pairs, the period and trips/h.
A few hundred cars, safe gaps, no overlaps or ghost cars	About 700 vehicles at the AM peak (cars, SUVs, vans, trucks of different lengths) using IDM car-following. Tests check that no two vehicles overlap; the gridlock counter stays at 0.
Junction right-of-way, no collisions or freezing	Only compatible movements share a junction, and cars cannot enter unless the exit has room. Junctions on arterials and downtown have fixed-time signals (green, 3 s yellow, 2 s all-red, right on red after stopping); suburban crossroads are all-way stops; at T-junctions only the ending street stops.
Route picks a longer, less congested path, both costs logged	A* over live link travel times at departure. The route panel shows the chosen route's cost next to the shorter route's, with distances; the route is drawn in yellow. Probe example: 2,790 m chosen (estimated 312 s) over a jammed 2,581 m route (estimated 1,931 s).

Cut or changed

- **Stack:** the spec planned plain JavaScript on a 2D canvas, with Three.js as a fallback. We used TypeScript and Three.js instead, because our approved prototype was already 3D in Three.js.
- **Junction control:** the plan was fixed right-of-way rules, which are still used. We added fixed-time signals and stop signs on top so junctions behave like real ones; downtown signals also let the city carry more traffic without gridlock. Adaptive signals are still cut.
- **Pulled forward:** two Could-haves, a non-grid street network (uneven blocks, cul-de-sacs, parks, a mall) and some 3D polish (day and night, brake lights, turn signals, a chase camera for the probe car).
- **Still simplified:** IDM uses textbook parameters, not calibrated ones, and there is no lane changing. Routes are fixed at departure, so a car does not reroute mid-trip; the probe's actual time (510 s) was longer than its estimate (312 s) for this reason.
- **Deferred as planned:** emergency incidents and dispatch, GIS import, multimodal traffic. The Swarm & Boids pillar is only partly shown: car-following is a local-rule agent model, and boids-style yielding to emergency vehicles comes with dispatch.

Performance and Stress Test

Target (Assignment 2)	60 FPS with a few hundred vehicles and about 200 road segments
Measured	[FPS] FPS at the AM peak with about 700 vehicles on about 1,100 street segments (2,194 directed links), five times the planned network. One simulation step at the peak takes 0.24 ms on the CPU, so the 60× speed setting (10 steps per frame) costs about 2.4 ms per frame.
Limit	The traffic model, not the frame rate, is the limit. At 13,000 trips/h (the default) the peak holds 697 vehicles with 0 gridlock removals. At 1.15× demand it reaches 893 vehicles with the first 2 removals; at 1.3× it reaches 1,236 vehicles, 16 removals and growing queues, while a step still costs only 0.37 ms. The stress test in the video opens the live build with `?demand=` above 1 to show this.

AI Utility

The spec listed Claude and ChatGPT for scaffolding the traffic model and for mockups; no AI-generated assets and no runtime AI were planned, and none are used (the simulation is classical algorithms only).

- **Saved time:** the first versions of the data structures, A*, the Three.js rendering (instanced cars, merged city geometry), the sidebar and the tests came together in days instead of weeks. AI was also quick at writing tests that reproduce a bug.
- **Cost time:** traffic behaviour needed our own testing. Examples: a yellow-light rule let stopped cars drive into the junction; signal tests first used more demand than a junction can carry, so they failed for the wrong reason; raising demand caused gridlock that took a design change (downtown signals, more even block sizes) to fix; an automated edit once overwrote part of a source file. Tuning demand and signal timing meant running the simulation many times and reading the numbers ourselves.
- **Overall:** clearly faster than without AI, but the time went into checking and tuning rather than writing code.

Roadmap to Final

- Incidents that spawn on the map (and can be placed by the user), plus road closures.
- Dispatch of the nearest available matching unit (ambulance, fire, police), with live rerouting around closures and congestion.
- Traffic yielding to emergency vehicles with boids-style local rules (the second pillar).
- Mutual-aid fallback, response-time and calls-handled metrics, and a fleet and incident panel.
- If time allows: rerouting mid-trip, multi-unit incidents and adaptive signals.

Links

Playable web build	mihirshinde-29.github.io/rapid_route (`?start=8` opens at the morning peak; any modern desktop browser)
Video	[ADD VIDEO LINK]
Source code	github.com/MihirShinde-29/rapid_route; build steps are in README.md (`cd sim`, `npm install`, `npm run dev`)